



Skin Cancer cells Classification

GIUSEPPE CALVARUSO

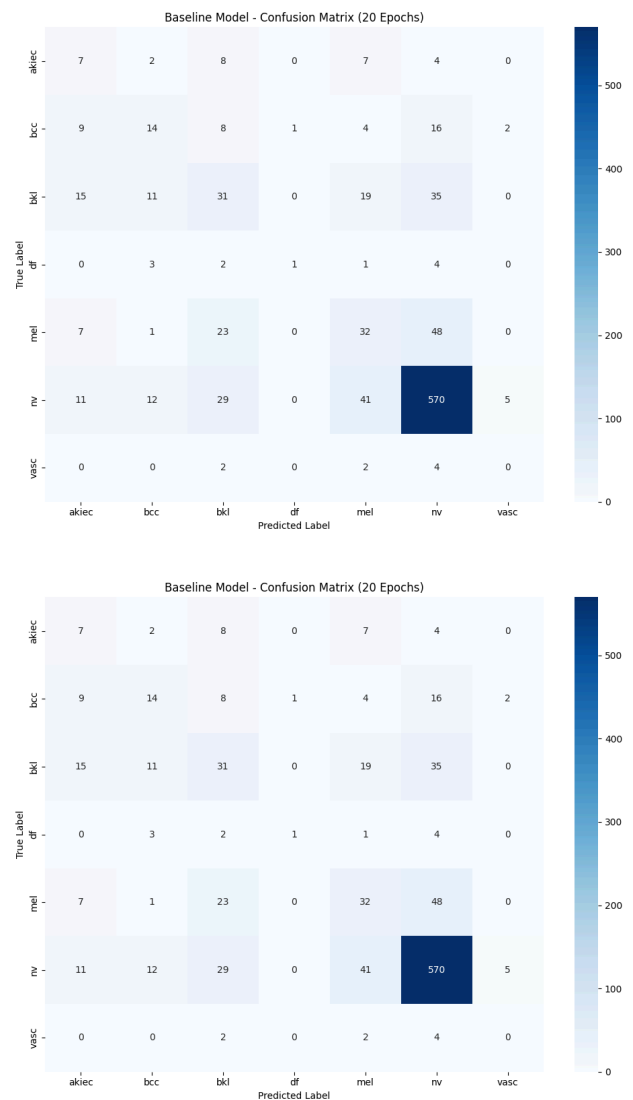
CLINIC PROBLEM AND BASELINE TRAP

The project objective is to classify through a CNN some dermoscopic images.

The main problem is the nature of the dataset is the unbalanced classes .
The nevi(nv) are 67% of the images, melanom (mel) only 11%.

As starting point, I've trained a simple CNN without optimization (see **1_Giuseppe_Calvaruso.py** file) .
The model has achieved 67% of Accuracy. In a medical environment, it is not a good result, we need a better recall over melanoms. The baseline model has under 30% recall

This is the accuracy paradox. In diagnostic, a False Positive (a precautionary biopsy) is tolerable, a false negative not (not diagnosed tumor)



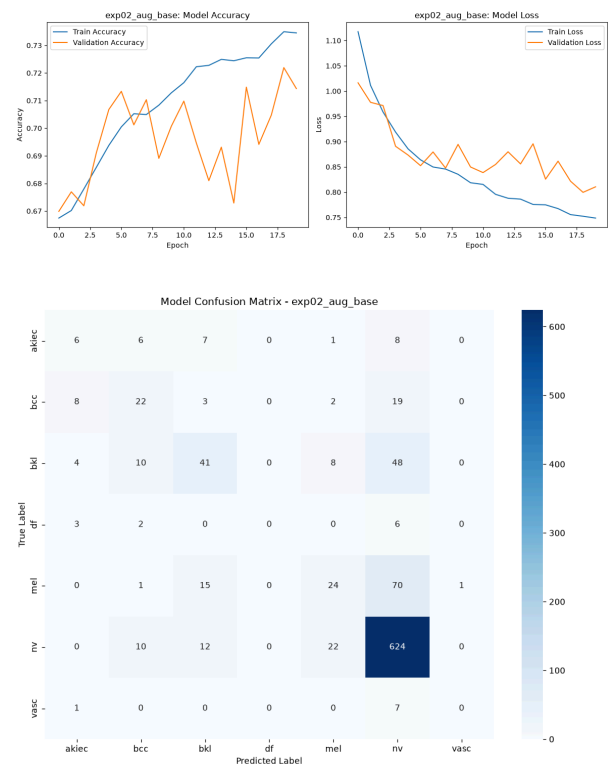
FIGHTING OVERFITTING (REGULARIZATION)

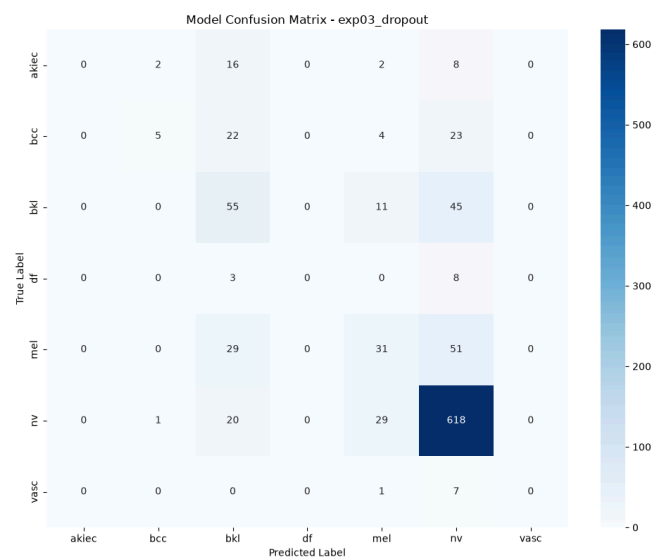
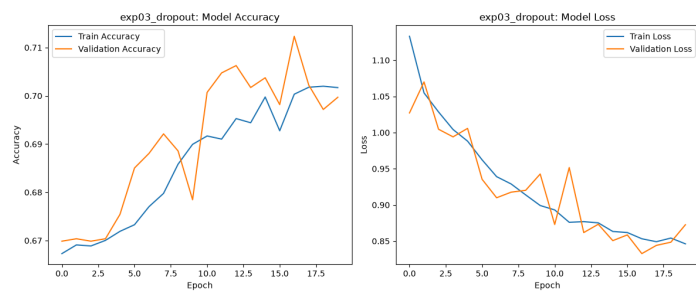
The first experiment after the baseline focuses on generalization. The baseline model tends to memorize the dataset and has generalization problems.

In the “**2_Data_augmentation.py**” experiment I used Data Augmentation. This approach uses the original dataset and creates artificially data starting from the original one using random flips, rotations and zooming.

Augmentation alone does not resolve the problem, the validation loss began increasing after few epochs

In the “**3_Dropout.py**”, I’ve used the dropout technique. This technique turn off, during training, about 50% of artificial neurons. I’ve used the output of experiment 2 as the base model for the dropout technique. The dropout experiment confirm the major problem: the unbalanced classes.

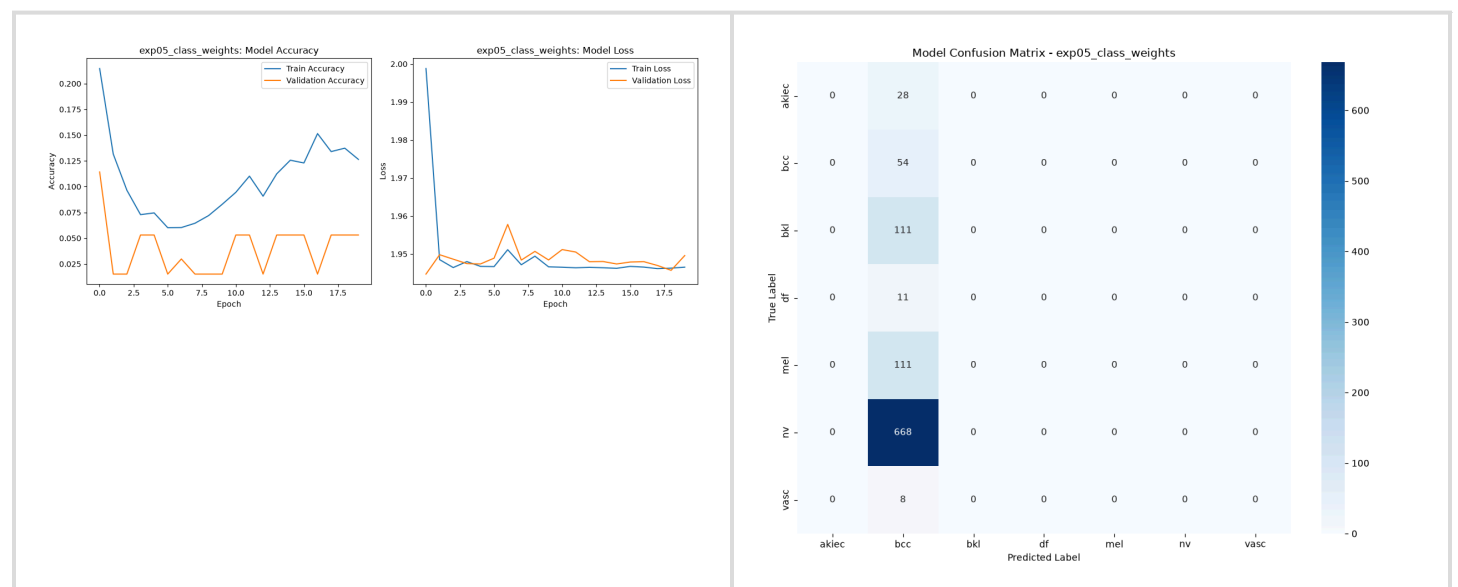
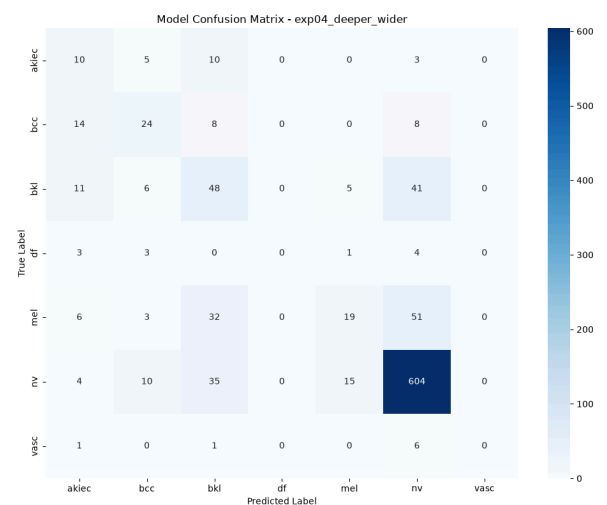
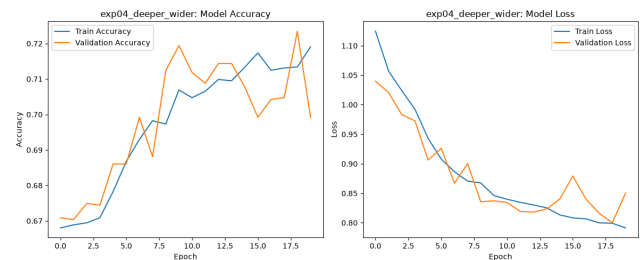




ARCHITECTURAL EXPANSION AND CLASS WEIGHT COLLAPSE

After building a strong model as starting point with dropout technique, the experiment **"4_Deep_and_Wide.py"** takes the previous model and enhances it. I've increased convolutional filters up to 128 and the dense layer up to 256 neurons, maintaining the data augmentation and dropout techniques. This allowed the building of a model that starts to recognize minority classes

In experiment 5 **"5_Class_Weights.py"**, I applied class weights on minority classes returning to standard 3-blocks training. The results was catastrophic. As graphs shows, there was a complete gradient collapse.



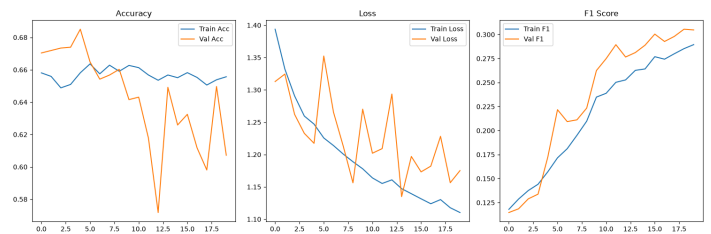
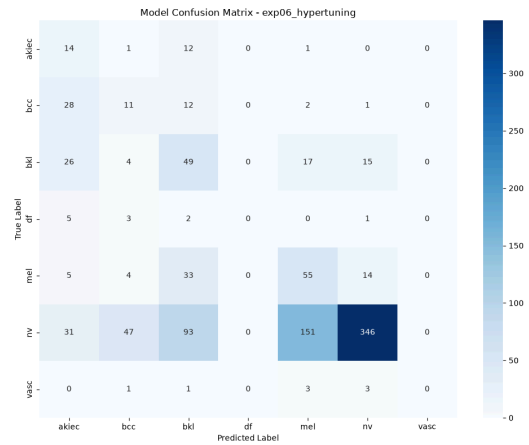
REFINEMENT: HYPERTUNING

Since experiment 5 collapses so I've decided to hypertune the dropout model in the

"6_Tuning_Dropout_model.py". In this model I've used a soft approach for weight penalties. I've applied a square-root penalty to cap, in order to the model to learn minority classes features, without forgetting the majority ones.

I've adjusted the Dropout to 0.4 and lowered the learning rate.

As the confusion matrix shows, the model successfully, the majority - class problem. The accuracy drop to 48%, but recall increases.



EXPERIMENT 7: CLINICAL TRIAGE STRATEGY

In this experiment

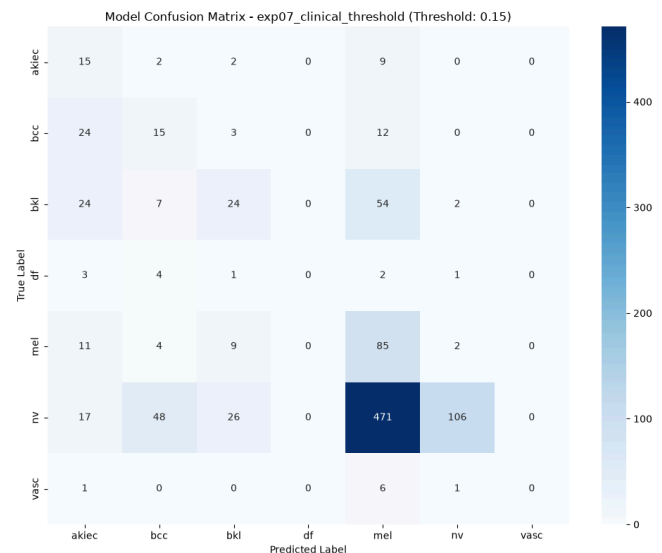
“7_Applying_real_clinical_threshold.py”, I’ve focused over patient safety than math accuracy.

Instead of continue to using the argmax function (choosing the class with highest probability), I extract the raw proability

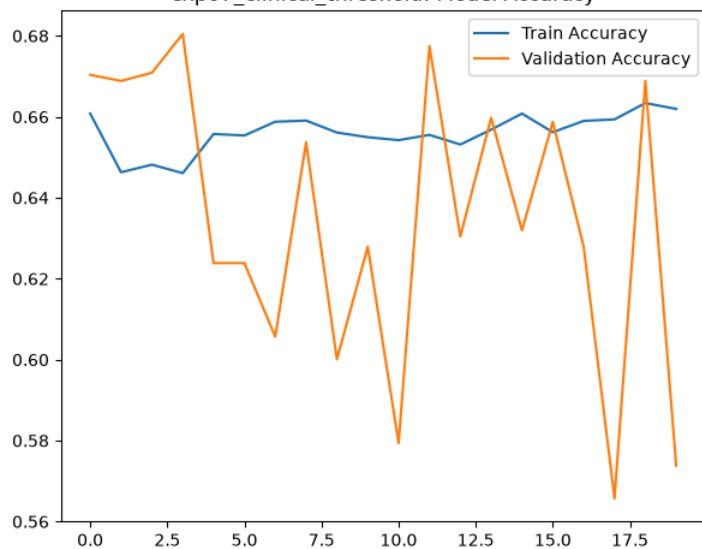
Melanoma : If the probability of prediction for melanoma is $> 15\%$, the image is treated as melanoma

In this way, more melanoma were detected.

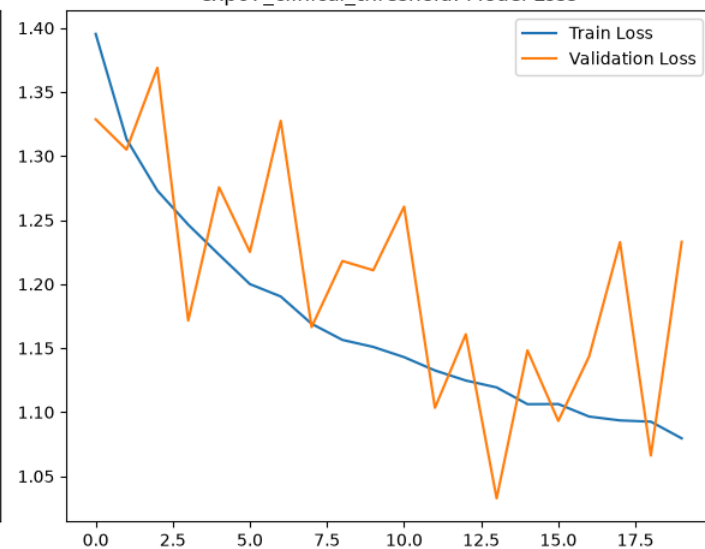
The global carrucary drops to 25% in order to do precautionary false positives.



exp07_clinical_threshold: Model Accuracy



exp07_clinical_threshold: Model Loss



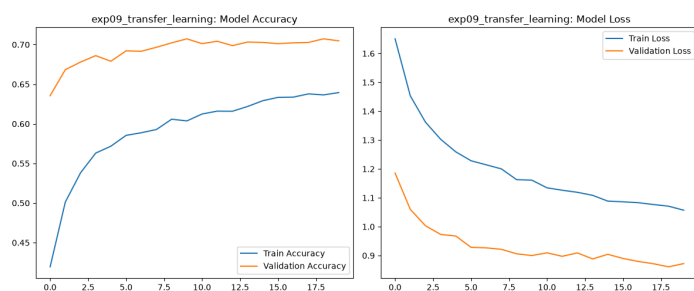
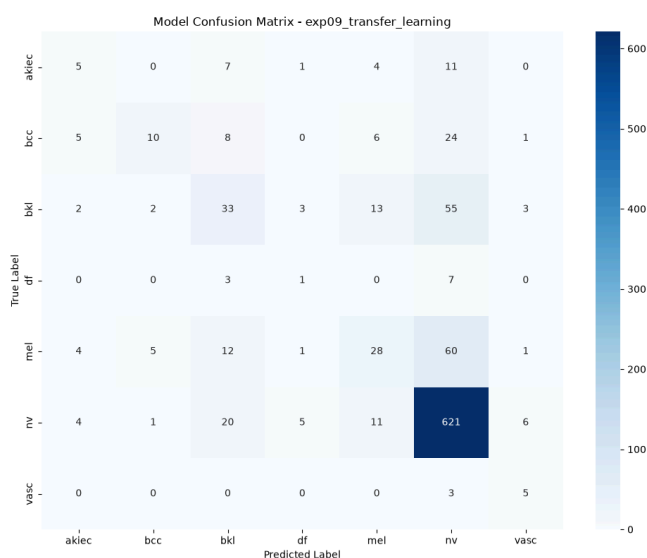
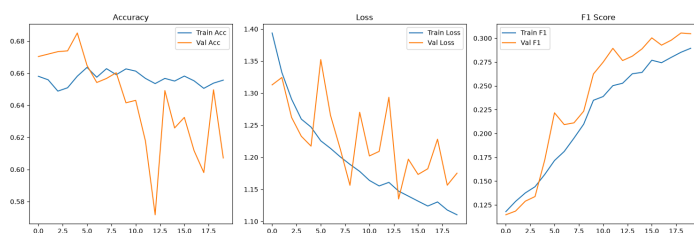
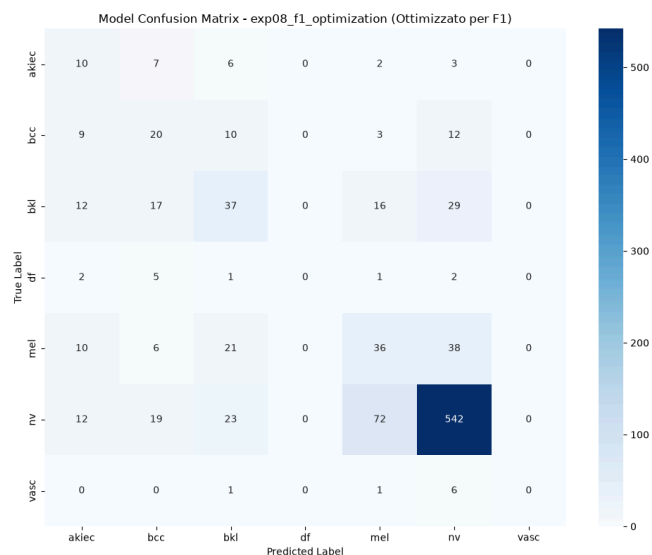
F1-Optimization and Transfer Learning Paradigm

On experiment

"8_Balanced_f1_model.py" I've focused on f1-Score improvements rather than accuracy. In this experiment I was searching balance over precision or recall. In this experiment the model had reached the feature-extraction limit

In order to build a more robust model, I've used the transfer learning **"9_Transfer_Learning.py"** with MobileNetV2. I've frozen the pre trained base weights and attached a GlobalAveragePooling2d classification head.

In this way, I reverted to accuracy trap for the problem



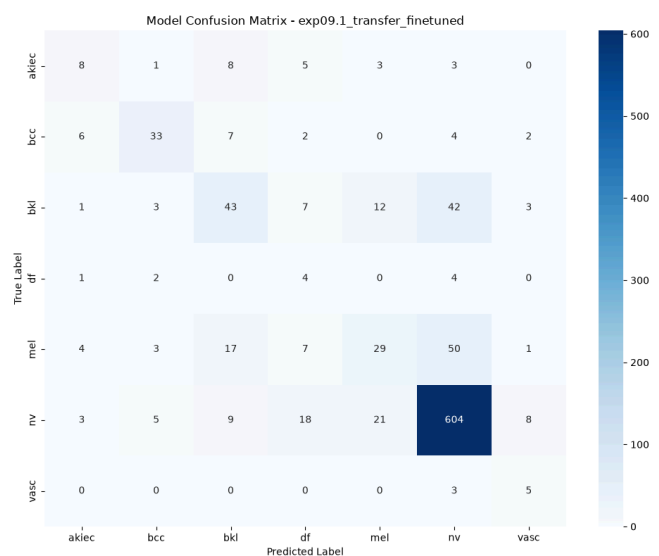
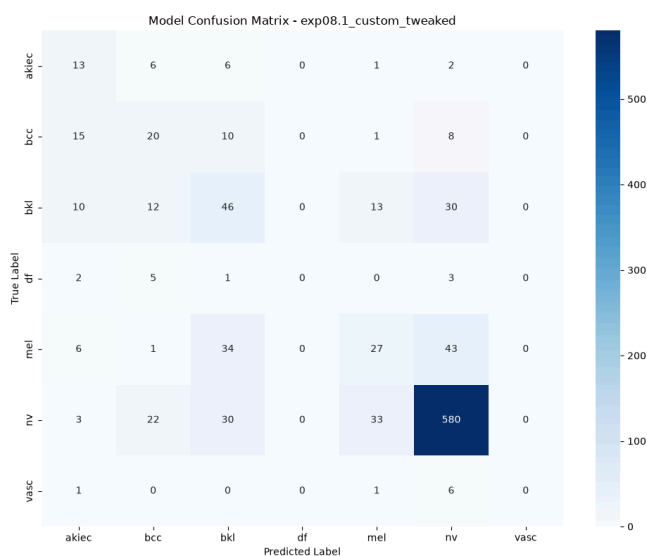
FINAL TWEAKS AND ADVANCED FINE-TUNING

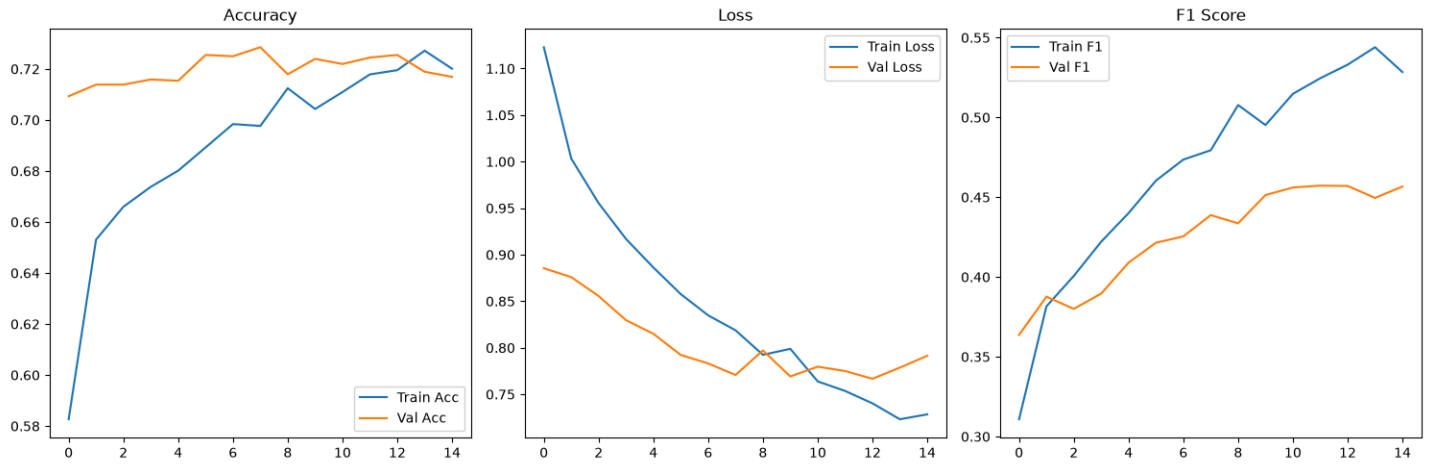
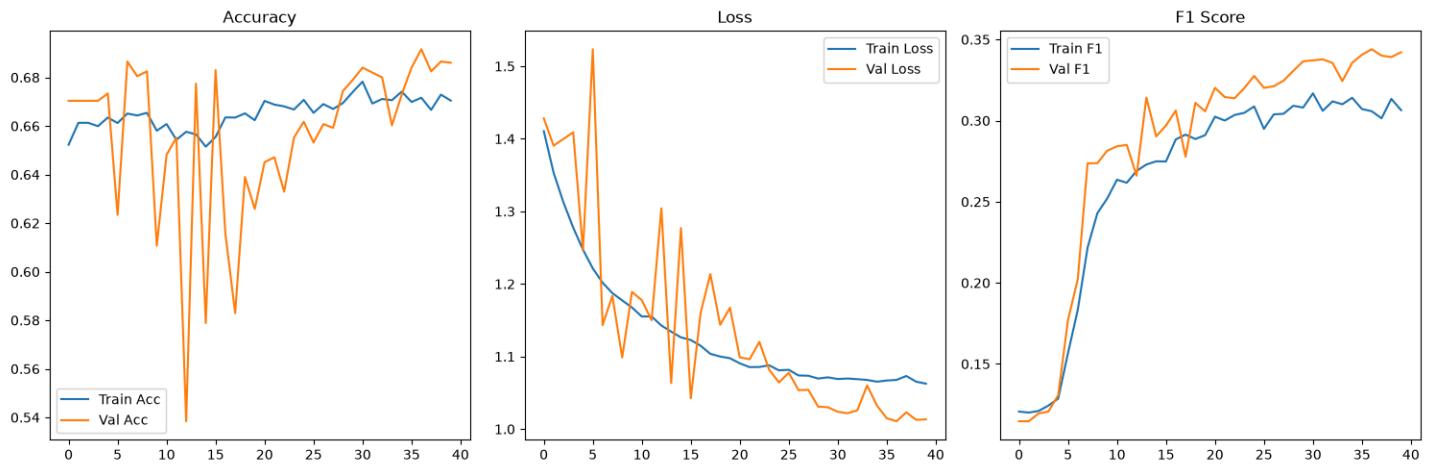
In experiment “**8.1_Balanced_f1_model_tweaked.py**” I’ve tweaked the balanced_f1_score model.

I’ve added some Random Contrast to data augmentation , added early stopping(halting learning after 8 epochs of no improvement) and added a ReduceLROnPlateau.

“**9.1_Transfer_Learning_model_tweaked.py**” experiment take the model based on mobile net and tweak it . I’ve unfrozen the first 30 layers in order to the model to adapt to the images.

I’ve introduced RandomContrast and ModelCheckpoint. The fine-tuned model improves. It relies not only over accuracy , but also confusion matrix shows an improvement over minority classes





(In the upper image the graph for experiment 8.1, in the down one the image for experiment 9.1)

The end of the Journey: Error Analysis- Understanding Models Limitations

In this analysis i will draw some conclusions about my dear models.

In
“8.1_Balanced_f1_model_tweaked.py” I pushed my custom model based over F1 introducing Random Contrast and some callbacks. The results shows that, even with tweaking, we have a lot of frequently misclassified **mel** and **bkl**

Then, I’ve started to tweak the transfer learning model on experiment

“9.1_Transfer_Learning_model_tweaked.py”. I’ve unfEven with tweaking, the error analysis shows the dataset unbalanced problem. I’ve introduced a microscopic learning rate In real world deployment this errors are unacceptable. A real diagnostic tool must have a reject option or a low-confidence flag.

Conclusion: Even if we integrate sophisticated algorithm, the model cannot invent data, and google model has no magic wand to resolve the problem

Model Classification Report - exp08.1_custom_tweaked
Tweak: EarlyStopping + LR Scheduler + Contrast Augmentation
Test Accuracy: 69.22%

	precision	recall	f1-score	support
<u>akiec</u>	0.26	0.46	0.33	28
<u>bcc</u>	0.30	0.37	0.33	54
<u>bkl</u>	0.36	0.41	0.39	111
<u>df</u>	0.00	0.00	0.00	11
<u>mel</u>	0.36	0.24	0.29	111
<u>nv</u>	0.86	0.87	0.87	668
<u>vasc</u>	0.00	0.00	0.00	8
accuracy			0.69	991
macro avg	0.31	0.34	0.32	991
weighted avg	0.69	0.69	0.69	991

Model Classification Report - exp09.1_transfer_finetuned
Tweak: MobileNetV2 Fine-Tuning (bottom 30 layers unfreezed) + LR 1e-5
Test Accuracy: 73.26%

	precision	recall	f1-score	support
<u>akiec</u>	0.35	0.29	0.31	28
<u>bcc</u>	0.70	0.61	0.65	54
<u>bkl</u>	0.51	0.39	0.44	111
<u>df</u>	0.09	0.36	0.15	11
<u>mel</u>	0.45	0.26	0.33	111
<u>nv</u>	0.85	0.90	0.88	668
<u>vasc</u>	0.26	0.62	0.37	8
accuracy			0.73	991
macro avg	0.46	0.49	0.45	991
weighted avg	0.73	0.73	0.73	991

CONCLUSION AND FUTURE DEVELOPMENTS

PROJECT SUMMARY

This project, with its wonderful failures, shows a real approach to computer vision, explaining the most important problem of the subject: the lack of data on real-world dataset.

I've started with a simple CNN and I've tried to systematically address overfitting, gradient collapse and imbalance.

Then I've shifted my focus over F1-score, because the health of the patient is more important than mathematical accuracy (important for a **triage medical application**)

Then I've realized that only with my knowledge, I wouldn't have succeeded on realizing a good model.

I've then integrated a MobileNetV2 architecture.

This creates a more robust classifier model that balances architectural power with strict clinical constraints.

FUTURE WORKS

Data acquisition :The primary bottleneck is the lack of data. Future iterations could be collaborating with clinics to sample minority classes like vasc and df to break data scarcity

Model Tweaking:

For future work, we can use the 8.1 and 9.1 model as starting point.

We can reduce the learning rate and increase the epoch and try different optimizer such as SGD.

For transfer learning model (9.1) we can use a progressive unfreezing strategy or use better model like ResNet50

Ensamble Architecture:

In the future I could integrate an Ensemble architecture merging the f-1 optimized model with deep texture recognition of transfer learning model.

Deployment

The MobileNetV2 is a lightweight model. In the future, it will be compiled through tensorflow lite. Then this can be installed as app in

mobile smartphone to have an offline triage assistant for general practitioners.

The on-line method is suggested in order to download update or enhance model with new data

As a infrastructure, we can use lambda by amazon AWS